

Chapter 2: Regularization

Basics and Applications of Deep Learning

Fall 2026

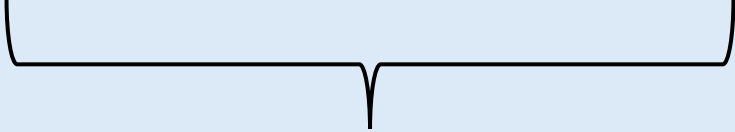
Department of Mathematical Data Science

Hanyang University ERICA

Lecturer: Sehyun Kwon



Regularization

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)$$


Data loss: Model predictions should match training data

Regularization

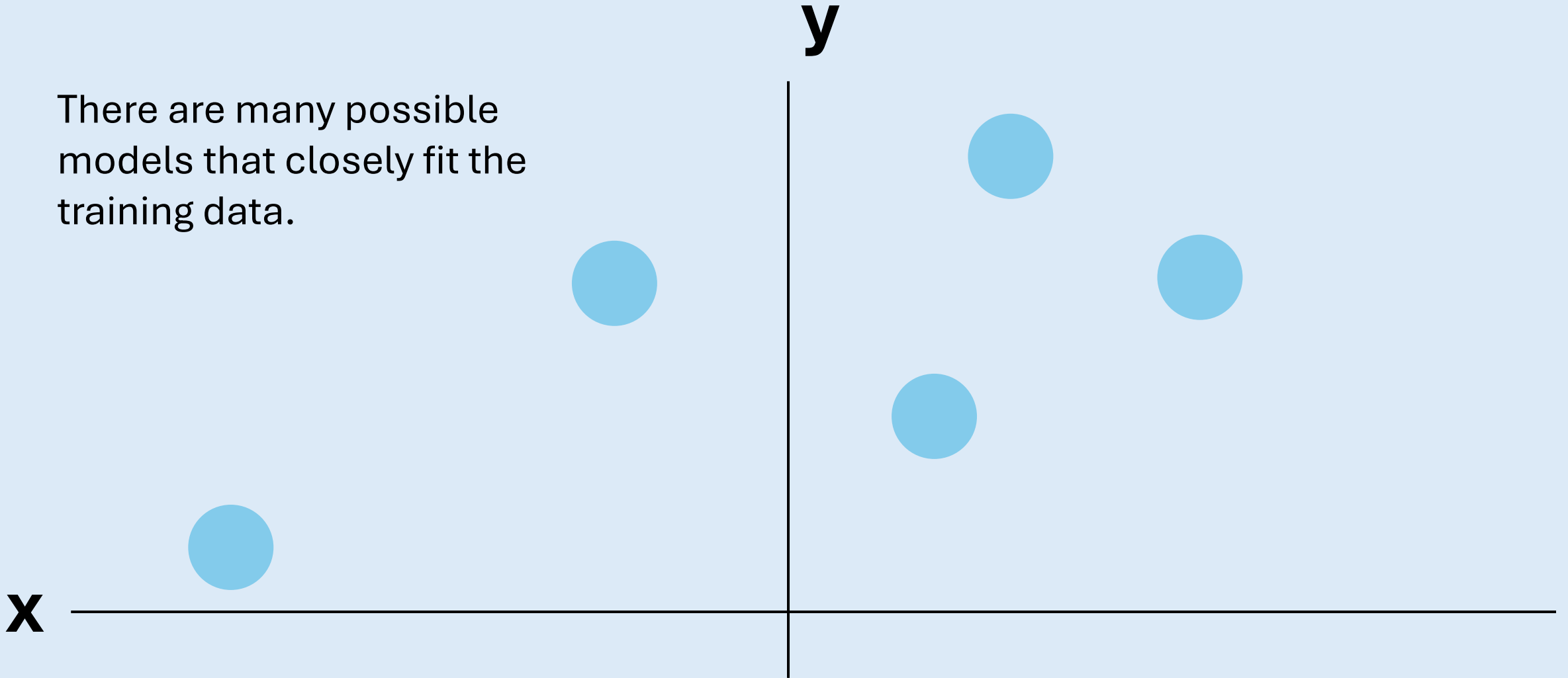
$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

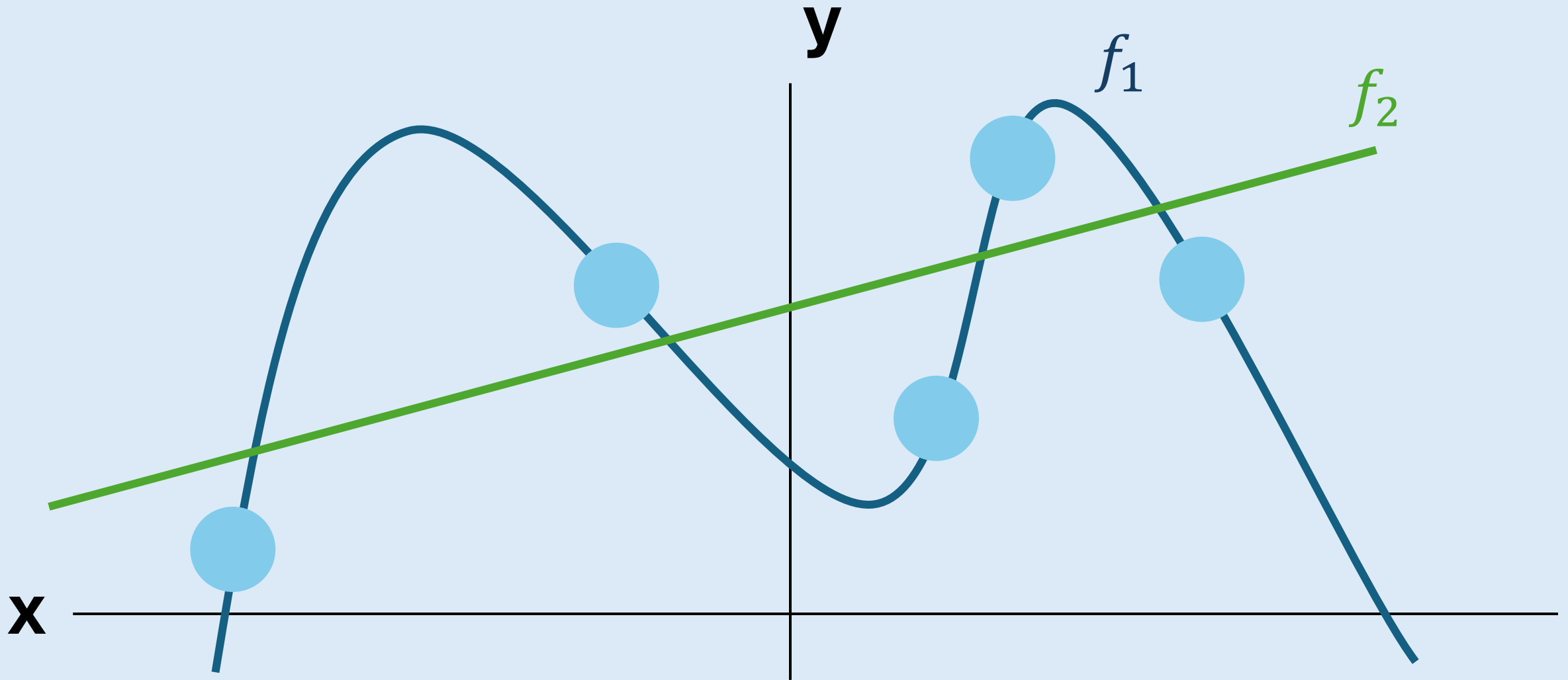
Regularization: Prevent the model from doing too well on training data

Regularization intuition

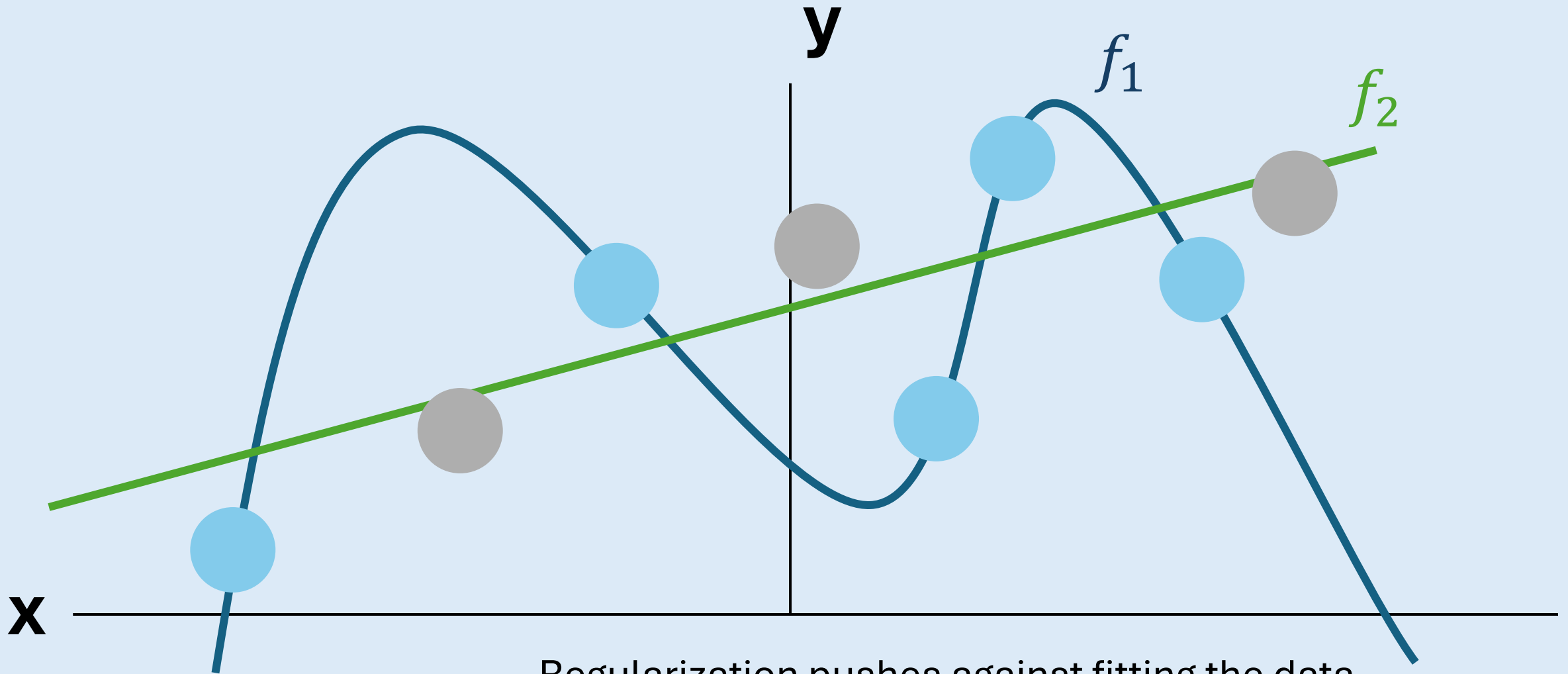
There are many possible models that closely fit the training data.



Regularization intuition



Regularization intuition



Regularization pushes against fitting the data too well so we don't fit noise in the data

Regularization

λ = regularization strength
(hyperparameter)

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing too well on training data



Occam's Razor: Among multiple competing hypotheses, the simplest is the best, William of Ockham 1285-1347

Regularization

λ = regularization strength
(hyperparameter)

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing too well on training data

How? Simple examples:

L2 regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1+L2): $R(W) = \sum_k \sum_l (\beta W_{k,l}^2 + |W_{k,l}|)$

← prevent weights from becoming too large.

Regularization

λ = regularization strength
(hyperparameter)

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing too well on training data

Simple examples

L2 regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1+L2): $R(W) = \sum_k \sum_l (\beta W_{k,l}^2 + |W_{k,l}|)$

Others (coming soon)

Dropout, Batch normalization, etc

Regularization

λ = regularization strength
(hyperparameter)

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing too well on training data

Why regularize?

- Express preferences over weights
- Make the model simple so it works on test data
- Improve optimization by adding curvature

Regularization: expressing preferences

$$x = [1, 1, 1, 1]$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

L2 Regularization

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

Which of w_1 or w_2 will the L2 regularizer prefer?

$$w_1^T x = w_2^T x = 1$$

Regularization: expressing preferences

$$x = [1, 1, 1, 1]$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

$$w_1^T x = w_2^T x = 1$$

L2 Regularization

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

Which of w_1 or w_2 will the L2 regularizer prefer?

L2 regularization likes to “spread out” the weights

Regularization: expressing preferences

$$x = [1, 1, 1, 1]$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

$$w_1^T x = w_2^T x = 1$$

L2 Regularization

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

Which of w_1 or w_2 will the L2 regularizer prefer?

L2 regularization likes to “spread out” the weights

Which one would L1 Regularization prefer?

Recap

- We have some **dataset** of (x,y)
- We have a **score function**: $s = f(x; W) \stackrel{\text{e.g.}}{=} Wx$
- We have a **loss function**:

$$L_i = -\log \left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}} \right) \quad \text{Softmax}$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + R(W) \quad \text{Full loss}$$

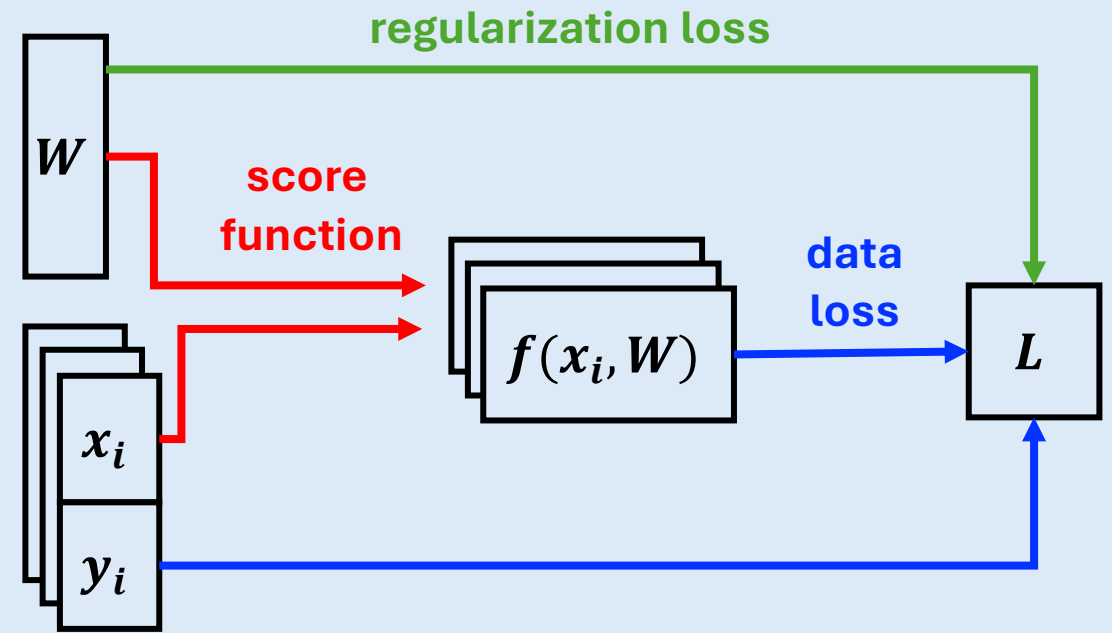
Recap

- We have some **dataset** of (x,y)
- We have a **score function**: $s = f(x; W) \stackrel{\text{e.g.}}{=} Wx$

- We have a **loss function**:

$$L_i = -\log \left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}} \right) \quad \text{Softmax}$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + R(W) \quad \text{Full loss}$$



Recap

- We have some **dataset** of (x,y)

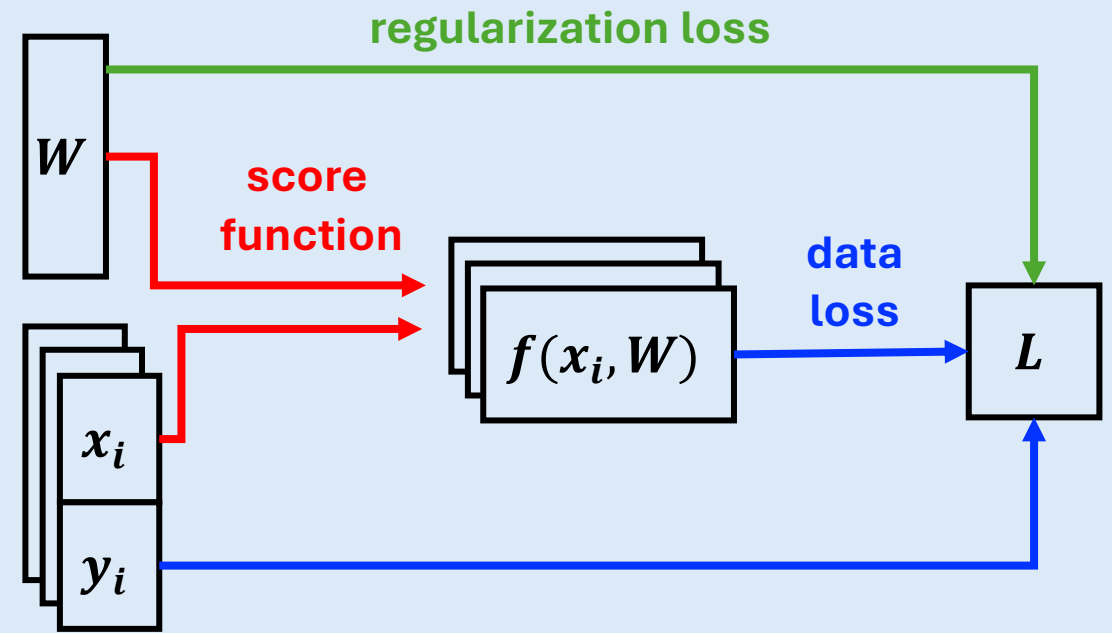
- We have a **score function**: $s = f(x; W) \stackrel{\text{e.g.}}{=} Wx$

- We have a **loss function**:

$$L_i = -\log \left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}} \right) \quad \text{Softmax}$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + R(W) \quad \text{Full loss}$$

Next question:
How do we find the best W ?



References

- Stanford CS231n
- Mathematical Foundations of Deep Neural Networks at Seoul National University.

